HubOptima: Combining Random Forest Demand Forecasting with a Matheuristic for E-Commerce Hub Location

CH.ARCHANA
Roll No. 23K91A6745
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

A.MEENAKSHI
Roll No. 23K91A6705
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

B.ARUN
Roll No. 23K91A6734
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

A.SAMPATH
Roll No. 23K91A6712
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

[GUIDE NAME]
[DESIGNATION], Project Guide
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

***Abstract*—**Hub networks of e-commerce carriers are usually planned on a fixed demand with exact solvers or simple rules, although demand moves between regions and seasons and exact solvers slow down as networks grow. This paper implements and measures a predict-then-optimise framework. A Random Forest forecasts regional demand from order history; a capacitated hub-location model is built from the forecast and solved by a matheuristic that prices hub sets with exact assignment programs and polishes the result with a kernel search; plans are scored on the demand that actually occurred and are re-made every month. On a synthetic dataset of 90 regions and 36 months, the forecaster reaches a MAPE of 7.67% against 17.6-19.7% for naive forecasters. For the October 2025 festival peak, the plan matches the exact optimum, costs 3.6-16.1% less than a clustering heuristic, a greedy heuristic and a largest-markets rule, and re-planning every month raises simulated fulfilment from 83.7% to 98.3% against a static plan. The matheuristic is 3.67% above the exact incumbent at 270 nodes, the peak was under-forecast by 13.2%, and the data are synthetic.

***Index Terms*—**demand forecasting, e-commerce logistics, hub location problem, matheuristic, mixed-integer programming, random forest, rolling-horizon re-optimisation.

I. INTRODUCTION

Electronic commerce depends on a physical network of hubs where parcels are consolidated, sorted and dispatched. Where the hubs are opened decides the fixed cost of the network, the distance parcels travel and how long deliveries take, which makes the hub location problem (HLP) a central problem of logistics planning [1], [6], [7]. The problem is combinatorial: even with 90 candidate sites, choosing six hubs and allocating every region to one of them has billions of solutions, and exact mixed-integer programming (MIP) is practical only for moderate networks [8], [9].

In practice, plans are made on a fixed demand, often an average of past months, by solving an exact model once, by applying a greedy or clustering heuristic, or by placing hubs at the largest markets. This costs money twice. The demand is wrong when the plan is used: in our October 2025 test a plan built on the historical average served only 66.0% of the orders that arrived. And the rules are not optimal: the largest-markets plan cost 16.1% more than the optimised one. The base paper [1] proposes to forecast demand with a Random Forest and to feed the forecast to a matheuristic, but it reports results on proprietary data, does not release code, and states the gain of the forecaster only as “X %”.

This paper presents HubOptima, an open implementation of that predict-then-optimise framework with a measured evaluation. The contributions are:

1) an end-to-end pipeline (cleaning, 19 features, Random Forest, capacitated hub MIP, matheuristic, evaluation) in about 2,000 lines of Python;
2) a matheuristic that screens swap, drop and add moves with a cheap estimate, prices the promising ones with exact assignment MIPs and finishes with a kernel search;
3) an evaluation protocol that plans on the forecast and scores on realised demand, so that forecast error appears as overloaded hubs, with five baselines and a rolling-horizon control;
4) a FastAPI and React application with 14 endpoints that exposes forecasting, planning, comparison and re-optimisation.

Section II reviews the literature, Section III the method, Section IV the results and Section V concludes.

A. System Hardware

All timings were measured on one desktop with an Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB of memory and Windows 11 Pro. No GPU was used and no other machine was tested.

B. Software Requirements

Python 3.11.9 with pandas 2.2.1, NumPy 1.26.4 and scikit-learn 1.4.2 [13] (Random Forest [11], k-means [18]); PuLP 3.3.2 [14] with the bundled CBC solver [15]; FastAPI 0.110 with uvicorn; React 18.3 and Vite 5.4 for the interface; and Node.js 24. All components are open source and the system runs offline.

II. LITERATURE SURVEY

A. Base Paper

Danach et al. [1] study the HLP for e-commerce with centralised hubs, dynamic demand, capacity limits and service levels. Their model minimises fixed hub cost,

variable transport cost and a penalty on unmet demand, with continuous flows, binary hubs and capacity levels. The solution approach is a matheuristic: a relaxed MILP gives an initial solution, Variable Neighbourhood Search (VNS) [16] improves it, a Random Forest trained on historical demand forecasts the demand at each location, and the forecasts re-optimise the hubs and are validated on peak, spike and disruption scenarios. On the proprietary data of a global transport company (two years of demand, thousands of locations) the paper reports delivery-time compliance of 92.3%, fulfilment of 97.6%, robustness of 0.87, improvements of 7.8%, 9.4% and 16% over traditional methods, cost reductions of 15-20% and networks of 50 to 500 nodes. The authors list sector-specific data, limited scalability testing, offline evaluation and unexplored learners as limitations. HubOptima differs in that it releases code and data, measures the forecaster against naive benchmarks, evaluates plans on realised demand, and replaces VNS by a screened local search with kernel search [17].

B. Related Work

Foundations. O'Kelly introduced interacting hub facilities [6]; Campbell et al. surveyed the problem family [7]; Ernst and Krishnamoorthy gave efficient algorithms for the uncapacitated single-allocation p-hub median [8]; Danach et al. combined Lagrangian relaxation with a hyper-heuristic for a capacitated single-allocation location-routing problem [9].

Metaheuristics. Zheng et al. locate drone logistics hubs with a diversity-based hybrid particle swarm optimiser, reporting a 42.58% efficiency gain over other algorithms in the abstract [2]. Dehghan Chenary et al. combine Pareto-based clustering with a metaheuristic for a bi-objective mobile hub problem with congestion and compare it with MOPSO and NSGA-II on Australian Post data [4].

Exact methods and uncertainty. Yang et al. model a maximal-detour liner-shipping p-hub centre with three MILPs and an improved Benders decomposition [3]. Zhang et al. formulate a two-stage adaptive robust hub location problem under budgeted demand uncertainty and solve it by column-and-constraint generation, reporting better out-of-sample performance than a stochastic model [5]. Kahalimoghadam et al. address the number and location of micro-consolidation centres under growing e-commerce demand [10].

Methods. Maniezzo et al. define matheuristics [12]; kernel search [17] and VNS [16] are the building blocks the base paper and this work draw on; Hyndman and Athanasopoulos give the evaluation conventions for forecasts [19]. Table I summarises the works; [2]-[5] are summarised from their abstracts only.

TABLE I
RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1]	RF forecast + matheuristic	Proprietary	92.3% / 97.6% / 0.87	Data not public
[2]	Hybrid PSO	Village drones	+42.58% (abstract)	No forecast
[3]	MILP + Benders	Liner shipping	Faster exact solve	Fixed demand
[4]	Pareto clustering	Australian Post	Beats MOPSO, NSGA-II	Demand given
[5]	Adaptive robust	Not stated	Better out-of-sample	Assumed set
[9]	Lagrangian + hyper-heur.	Not reviewed	As cited in [1]	No forecast

Research gap. Most studies take demand as given or assume an uncertainty set; the base paper's data, code and matheuristic parameters are unavailable; the effect of forecast error on the plan is not shown; and the value of monthly re-planning and the exact-versus-matheuristic trade-off as networks grow are not measured. HubOptima addresses these four points.

III. METHODOLOGY

A. Data

No public region-level order data was available, so the data are synthetic (seed 42): 90 regions (30 Indian cities and 60 satellite districts) and 36 months, 2023-01 to 2025-12, giving 3,240 region-months and 48.3 million orders. Monthly orders follow population x tier-specific penetration, a 1.0-2.2% monthly growth, festival factors (October 1.35, November 1.25), a promotion lift of 14% and 6% noise; the network peaks at 2.38 million orders in October 2025. Loading removes duplicates, interpolates gaps and sorts; the generated data has no gaps, so these guards serve real files. The Random Forest learns a process that the authors wrote down, so its accuracy is not evidence of accuracy on real orders.

B. Demand Forecasting

For every region and month, 19 features are built: lags 1, 2, 3, 6 and 12, rolling means over 3 and 6 months and a rolling standard deviation (all shifted one month so that a month never sees itself), a trend, month, quarter, a festival flag, the promotion flags, population, tier, penetration, order value and return rate. A Random Forest regressor [11] with 400 trees, minimum leaf size 2 and 60% of features per split is trained on 2023-04 to 2025-06 (2,430 rows) and tested on the six months 2025-07 to 2025-12 (540 rows); there is no tuning and no cross-validation. A past month is forecast using only earlier data, and the next month is forecast from the lags of the latest month; the forest is a single global model for all regions, so small regions borrow strength from large ones. Training takes 0.8 s. Fig. 1 shows the pipeline and Fig. 2 the software.

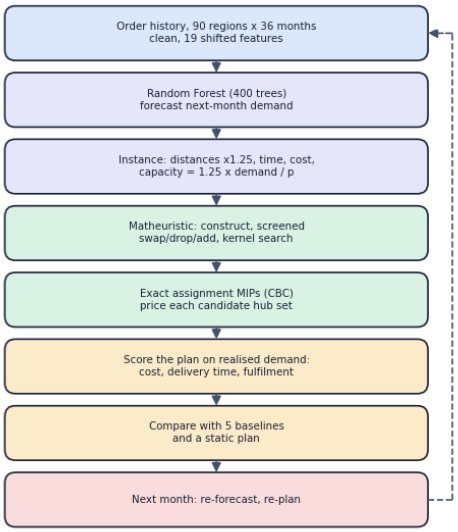


Fig. 1. Predict-then-optimise flow of HubOptima.

C. Hub-Location Model

Every region i is a demand point with planned demand D_i (the forecast) and a candidate hub site j. Distance is 1.25 times the haversine distance [20], delivery time is t_ij = d_ij / 45 km/h + 6 h (3 h for local delivery), the unit cost is c_ij = 0.04 d_ij + 1.5 t_ij (INR per order), the fixed cost is INR 2.5 million per month scaled by tier (1.35, 1.0, 0.75) and the capacity of a hub is C = 1.25 SUM_i D_i / p with p = 6. With binary y_j (hub open) and x_ij (region i served by j):

$$\min \ \Sigma_j f_j y_j + \Sigma_i \Sigma_j c_{ij} D_i x_{ij} \qquad (1)$$

$$\text{s.t.} \ \Sigma_j x_{ij} = 1 \ \ \forall i; \ \ x_{ij} \le y_j; \ \ \Sigma_j y_j \le p \qquad (2)$$

$$\Sigma_i D_i x_{ij} \le C \, y_j \ \ \forall j \qquad (3)$$

An optional hard constraint x_ij = 0 whenever t_ij exceeds the 48 h SLA is available; by default the SLA is soft, so late deliveries are penalised through the time term and reported by a reliability indicator instead of being forbidden. A hard SLA can make the problem infeasible, and the API then returns an explanation instead of a plan. Compared with [1] the model is single-allocation with one capacity level, no unmet-demand variable and no robust objective: the role of the unmet-demand penalty is played by the evaluation step below, in which orders above a hub's capacity count as unfulfilled.

D. Matheuristic

Fixing the hub set leaves an assignment MIP of n x p variables (540 for six hubs), which CBC solves exactly, whereas the full model has n + n^2 binaries (8,190 for 90 regions). The matheuristic therefore searches over hub sets [12]. (1) A greedy weighted k-medoids start set is built and priced. (2) Swap, drop and add moves are generated for the closed candidates ranked by nearby demand; every move is estimated by a capacity-aware greedy allocation, moves more than 6% above the current cost are skipped, and the rest are priced by the exact assignment MIP with a cache; the first strictly better plan is accepted and the search restarts, until no move helps or 70% of the time limit is used. (3) A kernel search [17] solves the full MIP restricted to about 24 promising hubs: the open hubs, their nearest neighbours, the best-priced hub sets seen and the most attractive unexamined sites. For n = 90 and p = 6 a pass generates up to 138 moves (132 swaps, 6 drops); in the reported runs 14 to 18 assignment MIPs were solved per search, so most moves are rejected by the estimate. If the start set is infeasible, hubs are added by demand until it is feasible.

E. Evaluation Protocol and Implementation

Plans are made on the forecast and scored on the realised demand R_i. With L_j the realised load of hub j and a(i) the hub of region i, the share a hub serves is s_j = min(1, C / L_j), and

$$\textit{fulfilment} = \Sigma_i \, s_{a(i)} \, R_i \, / \, \Sigma_i \, R_i \qquad (4)$$

$$\textit{reliability} = \Sigma_i \, 1[t_{i,a(i)} \le 48 \text{ h}] \, s_{a(i)} \, R_i \, / \, \Sigma_i \, R_i \qquad (5)$$

Together with total cost, average delivery time and utilisation L_j / C these form the KPIs. They are simulated capacity measures, not observed deliveries. Baselines on the same forecast are a greedy heuristic, demand-weighted k-means [18], the largest-markets rule and the exact MIP (30 s limit); a fifth plan uses the historical average instead of the forecast. A rolling horizon re-forecasts and re-plans for each month from July to December 2025 and is compared with a static plan fixed on the earlier average; the forest is not retrained during the simulation. A scalability benchmark splits every region into 1, 2 or 3 nearby demand points to give 90, 180 and 270 nodes, with an exact-MIP time limit. The core package (about 1,000 lines) is called by a FastAPI layer with 14 endpoints, pydantic validation (for example p between 1 and 20, time limit 2 to 300 s) and background jobs, and by a React application with seven pages (Fig. 2). Data, model and results are plain CSV, joblib and JSON files, there is no database, and the random seed is fixed, so a run can be repeated. The command line, the original Flask interface and the API call the same functions, which keeps their numbers identical. The code was checked by four test functions (data, forecast contract, optimisation constraints including the 2% agreement of matheuristic and exact MIP, and the web pages) and an 18-check API smoke test, all of which pass.

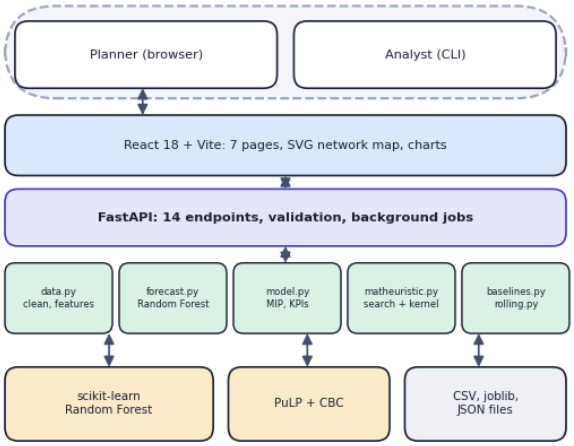


Fig. 2. Architecture: interface, API layer, core package, solvers and files.

IV. RESULTS

A. Forecast Accuracy

On the 540 held-out region-months the Random Forest reaches MAE 1,984, RMSE 6,619, MAPE 7.67% and R^2 0.9766, against MAE 4,151 and MAPE 19.71% for last month's value and MAE 3,786 and MAPE 17.56% for the 3-month moving average. The most important features are the 3-month mean (0.379), last month (0.242) and population (0.155). Network-level errors are 1.5-6.0% in ordinary months but 13.1% in October 2025 (an under-forecast) and 10.2% in December; the October volume of 2.38 million exceeds every earlier month (previous maximum 1.96 million), which a forest cannot extrapolate, a likely but untested cause. In December the median regional error is 7.0%, the mean 8.9% and 8 of 90 regions exceed 20%; the three worst, Pune (39.9%), Ahmedabad (39.3%) and Chennai (39.0%), are all over-forecasts, and their cause was not investigated.

B. Comparison with Traditional Methods

Table II gives the October 2025 plans for p = 6 scored on the 2,383,534 realised orders (forecast 2,067,718).

TABLE II
OCTOBER 2025 PLANS (P = 6, SCORED ON REALISED DEMAND)

Method	Cost (INR M)	vs prop.	Avg h	Fulf. %	Solve s
RF + matheuristic	62.2	-	8.56	92.3	4.89
RF + exact MIP	62.2	+0.0%	8.56	92.3	7.99
RF + greedy	69.3	+11.5%	9.47	89.3	0.01
RF + k-means	64.4	+3.6%	9.19	91.8	0.11
Largest markets	72.2	+16.1%	8.96	91.1	0.00
Hist. avg, no RF	67.2	+8.1%	9.91	66.0	3.60

The matheuristic and the exact MIP open the same six hubs (Mumbai, Delhi, Kolkata, Hyderabad-W2, Bengaluru-SE, Ahmedabad-SE) at INR 62.2 M per month; CBC proved optimality in 7.99 s against 4.89 s. Greedy, k-means and the largest-markets rule cost 11.5%, 3.6% and 16.1% more, and planning without a forecast costs 8.1% more and serves only 66.0% of orders. Because the peak was under-forecast, every forecast-based plan has three overloaded hubs and 89-92% fulfilment (Fig. 3, Fig. 4): Delhi runs at 122%, Bengaluru-SE at 111% and Mumbai at 110%. The plan costs INR 15.8 M in fixed hub cost, 15.8 M in transport and 30.6 M in delivery-time penalty, with an average delivery time of 8.56 h, a maximum of 37.7 h and an average distance of 166 km; Hyderabad-W2 (59%), Ahmedabad-SE (71%) and Kolkata (80%) stay below capacity. Greedy swaps Ahmedabad-SE for Nagpur-N, k-means opens Warangal, and the largest-markets rule has a longest delivery of 50.0 h, beyond the 48 h SLA. These are single-instance results without repeated runs.

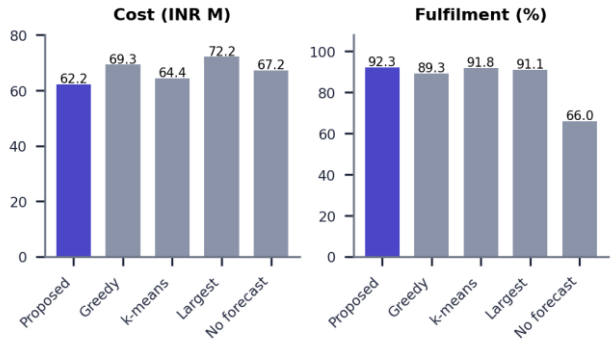


Fig. 3. Cost and simulated fulfilment by method, October 2025.

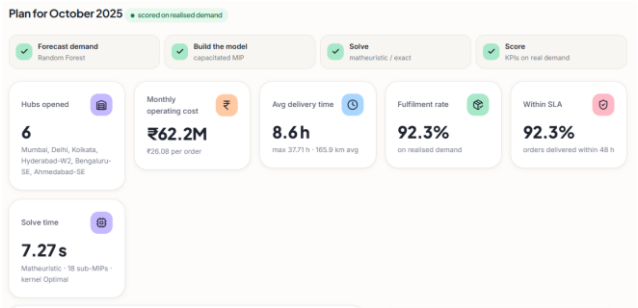


Fig. 4. Output screen: the October 2025 plan (6 hubs, INR 62.2 M, 92.3%).

C. Rolling Re-Optimisation and Scalability

Re-planning every month from July to December 2025 gives 98.3% average fulfilment against 83.7% for a static plan (+14.6 points), an average delivery time of 8.93 h against 9.98 h, and 4.9% lower average cost, with one hub change (Kolkata-W replaced by Kolkata in October). In the October peak the gap is widest, 92.4% against 64.9% (Table III, Fig. 6). The static plan is a simple control, and a generously sized static plan or monthly retraining was not tested. At 90 and 180 nodes the matheuristic equals the exact MIP (gap 0.00% and -0.05%) and is 4.1 and 5.2 times faster (3.1 s against 12.7 s; 8.3 s against 42.9 s). At 270 nodes (73,170 binary variables) CBC stopped without proof and the matheuristic (30.5 s) is 3.67% above its incumbent.

TABLE III
ROLLING RE-OPTIMISATION, JULY TO DECEMBER 2025

Month	Forecast error	Adaptive fulf. %	Static fulf. %	Hub change
2025-07	3.1%	100.0	95.3	-
2025-08	3.5%	99.4	88.5	-
2025-09	6.0%	99.0	93.3	-
2025-10	13.1%	92.4	64.9	+Kolkata / -Kolkata-W
2025-11	1.5%	98.7	70.8	-
2025-12	10.2%	100.0	89.5	-
Average	-	98.3	83.7	1 opening

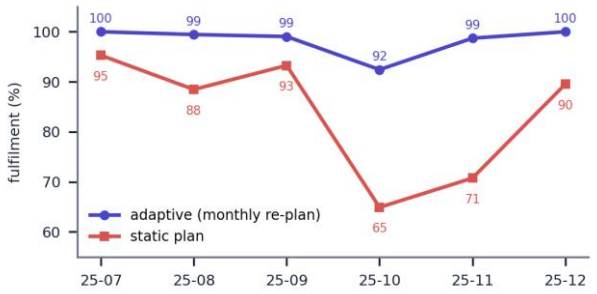


Fig. 5. Fulfilment of the adaptive and the static plan by month.

D. Comparison with the Base Paper

The datasets and metric definitions differ, so there is no head-to-head comparison. The base paper [1] reports compliance of 92.3% and fulfilment of 97.6% on proprietary data and robustness of 0.87; this work reports simulated fulfilment of 92.3% in the peak and 98.3% with monthly re-planning on synthetic data, and does not implement robustness. The base paper reports cost reductions of 15-20%; the reductions here, 3.6-16.1% against heuristics and rules and 0% against the exact MIP, point the same way against different baselines. It tests 50-500 nodes; this work tests 90-270.

E. Failure Cases and Observed Limits

The weakest result is the October peak, where a 13.2% under-forecast leaves 7.7% of orders unserved because capacity is sized from the forecast and the optimiser has no margin. At 90 nodes the matheuristic gives no practical gain over CBC: end-to-end planning took 12.2 s against 15.3 s for the exact MIP, and the UI showed 9.23 s against 9.29 s. At 270 nodes it is 3.67% worse than the incumbent. With six hubs, Dehradun-N is served 1,427 km away (37.7 h). The input API rejects n = 0 hubs (HTTP 422) and unknown months (HTTP 400), and the 4 tests and 18 API checks pass; no user study, load test or authentication test was done, and Fig. 5 shows the comparison screen.



Fig. 6. Output screen: six methods scored on realised demand.

F. Discussion

Three results are robust to the choice of dataset because they follow from the structure of the problem: a plan built on a forecast beats one built on an average when demand moves; an optimised hub set beats rules that ignore the cost structure; and capacity sized from a forecast is only as good as the forecast. Three others depend on the synthetic data and should be re-measured on real orders: the absolute forecast error, the absolute cost differences, which come from assumed cost parameters, and the size of the re-planning gain, which depends on how strongly the generated demand moves. The matheuristic's value is in scale, not at 90 nodes, and the 3.67% gap shows the price of the kernel restriction when CBC cannot finish. Planning for six hubs in about ten seconds on a desktop suggests the tool is usable interactively; whether planners find it so was not tested.

G. Threats to Validity

Internal: the hyper-parameters were fixed once and not tuned, and the solver times vary by several seconds between runs because of CBC and processor load. The forest was trained on data up to June 2025, so forecasts for earlier months are partly in-sample and are not used in any reported result. External: the demand process, the cost parameters and the SLA are assumptions; fulfilment and reliability are simulated; the comparison covers one month, the rolling horizon six months and the benchmark three network sizes. Construct: the base paper's compliance, fulfilment and robustness are defined on its own data and are not the quantities measured here.

V. CONCLUSION AND FUTURE SCOPE

HubOptima implements and measures the predict-then-optimise framework of [1]. Against the objectives, the data are cleaned and featurised, the Random Forest cuts the MAPE from 17.6-19.7% to 7.67%, the capacitated model includes cost, time and capacity, the matheuristic reaches the exact optimum at 90 and 180 nodes, monthly re-planning raises simulated fulfilment by 14.6 points, and five baselines are beaten on cost. The key finding is that forecast error becomes capacity overload, so the plan needs a margin. The limits are synthetic data, one split, no robustness analysis, a 3.67% gap at 270 nodes and no user study.

Future work: pilot on real orders; a safety margin or scenario objective; monthly retraining and other learners; VNS and multiple capacity levels; and a database with authentication for shared use.

REFERENCES

[1] K. Danach, A. Rammal, I. Moukadem, H. Harb, and A. Nasser, "Advanced optimization in e-commerce logistics: Combining matheuristics with random forests for hub location efficiency," IEEE Access, vol. 13, pp. 55915-55926, 2025, doi: 10.1109/ACCESS.2025.3550560.

[2] L. Zheng, G. Xu, and W. Chen, "Using improved particle swarm optimization algorithm for location problem of drone logistics hub," Computers, Materials & Continua, vol. 78, no. 1, pp. 935-957, 2024, doi: 10.32604/cmc.2023.046006.

[3] L. Yang, J. Zheng, J. Wang, and X. Hu, "The maximal detour liner shipping hub location problem: Improving the applicability of the p-hub center problem," Transportation Research Part E: Logistics and Transportation Review, vol. 179, Art. no. 103330, Nov. 2023, doi: 10.1016/j.tre.2023.103330.

[4] M. Dehghan Chenary, A. Ferdowsi, and R. F. Hartl, "A Pareto-based clustering approach for solving a bi-objective mobile hub location

problem with congestion," Logistics, vol. 8, no. 4, Art. no. 130, Dec. 2024, doi: 10.3390/logistics8040130.

[5] H. Zhang, J.-C. Wang, and Y. Gao, "Two-stage adaptive robust hub location problem under demand uncertainty," Naval Research Logistics, Dec. 2025, doi: 10.1002/nav.70043.

[6] M. E. O'Kelly, "The location of interacting hub facilities," Transportation Science, vol. 20, no. 2, pp. 92-106, 1986.

[7] J. F. Campbell, A. T. Ernst, and M. Krishnamoorthy, "Hub location problems," in Facility Location: Applications and Theory. Berlin, Germany: Springer, 2002, pp. 373-407.

[8] A. T. Ernst and M. Krishnamoorthy, "Efficient algorithms for the uncapacitated single allocation p-hub median problem," Location Science, vol. 4, no. 3, pp. 139-154, Oct. 1996.

[9] K. Danach, S. Gelareh, and R. Neamatian Monemi, "The capacitated single-allocation p-hub location routing problem: A Lagrangian relaxation and a hyper-heuristic approach," EURO Journal on Transportation and Logistics, vol. 8, no. 5, pp. 597-631, Dec. 2019.

[10] M. Kahalimoghadam, R. G. Thompson, and A. Rajabifard, "Determining the number and location of micro-consolidation centres as a solution to growing e-commerce demand," Journal of Transport Geography, vol. 117, Art. no. 103875, May 2024.

[11] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001.

[12] V. Maniezzo, M. A. Boschetti, and T. Stuetzle, Matheuristics: Algorithms and Implementations. Cham, Switzerland: Springer, 2021.

[13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[14] S. Mitchell, M. O'Sullivan, and I. Dunning, "PuLP: A linear programming toolkit for Python," Dept. of Engineering Science, University of Auckland, 2011.

[15] J. Forrest and R. Lougee-Heimer, "CBC user guide," in Emerging Theory, Methods, and Applications, INFORMS TutORials in Operations Research, 2005, pp. 257-277.

[16] P. Hansen and N. Mladenovic, "Variable neighborhood search: Principles and applications," European Journal of Operational Research, vol. 130, no. 3, pp. 449-467, May 2001.

[17] E. Angelelli, R. Mansini, and M. G. Speranza, "Kernel search: A general heuristic for the multi-dimensional knapsack problem," Computers & Operations Research, vol. 37, no. 11, pp. 2017-2026, Nov. 2010.

[18] S. P. Lloyd, "Least squares quantization in PCM," IEEE Transactions on Information Theory, vol. 28, no. 2, pp. 129-137, Mar. 1982.

[19] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.

[20] R. W. Sinnott, "Virtues of the haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.